ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



HỌC PHẦN: AN TOÀN PHẦN MỀM

ĐỀ TÀI: FUZZING WHITE BOX TESTING CHO QUẢN LÝ DÒNG HỌ

Giảng viên: ThS. Vũ Quang Dũng

Nhóm 5:

4. Lê Thị Kiều Trang (23010502 – K17 – KTPM(EL)_2)

Lớp học phần: CSE703093-1-1-26(N02)

Hà Nội, tháng 10 năm 2026

DANH SÁCH PHÂN CÔNG

STT	Mã sinh viên	Họ và tên	Công việc
4	23010502	Lê Thị Kiều Trang	Phần 3.7 (Kiểm thử bổ sung, phạm vi backend) Phần 4.6, 4.7 (Phân tích code lỗi WEB-01 về phân quyền, WEB-02 trong schema) Phần 5.3 (Những giới hạn của thực nghiệm) Phần 5.5 (Hướng phát triển fuzzing) Phần 6 (Kết luận) Phụ lục B, C.

3. Phương pháp thực hiện

3.7. Kiểm thử bổ sung và phạm vi backend

tools/probe.c đổi tên main khi include tệp C gốc, tạo một ca nền hợp lệ rồi gọi execute_clan_ genealogy_module. Các nhóm đầu vào gồm audit -1, 0, 24, 25; fund 16384, 16385, 32768; tiểu sử 100% dong gop; chỉ số phối ngẫu 14; thao tác vòng đời 3; và đo sizeof. Mỗi lệnh chạy trong một tiến trình mới, sau đó giải phóng các nút nếu tiến trình hoàn tất.

Một kiểm thử đơn vị riêng gán trace S00 → S15 rồi gọi record_execution_path để kiểm tra cơ chế so khớp. Trace này được dựng chủ động cho bộ đo; không được báo cáo như một đường đi thực thi tự nhiên của module.

Các tệp frontend/backend không thay đổi so với bản trước, được xác định bằng đối chiếu nội dung. Nhận xét phân quyền vì vậy được giữ và kiểm tra lại vị trí mã. Kiểm thử runSchema với kết nối giả được chạy lại, không kết nối MySQL và không xác nhận ứng dụng web đã chạy tích hợp.

4. Kết quả và phân tích

4.6. Vấn đề WEB-01 về phân quyền theo dòng họ

Mức bằng chứng: phân tích tĩnh; chưa thử nghiệm HTTP tích hợp.

Trong backend/src/routes/members.js, route danh sách sử dụng điều kiện clanId = ? lấy từ req.user.clanId. Tuy nhiên, route xem chi tiết ở dòng 55–65 truy vấn thành viên chỉ theo id. Các thao tác sửa và xóa cũng có truy vấn dựa trên id mà không đồng thời ràng buộc clanId của người dùng.

Đoạn mã tiêu biểu của route chi tiết:

router.get("/:id", async (req, res) => { try {

```
    const [rows] = await pool.query(
      "SELECT * FROM tbl_members WHERE id = ?",
      [req.params.id]
    );
    if (!rows.length) {
      return res.status(404).json({ error: "Không tìm thấy thành viên." });
    }
    return res.json(rows[0]);
  } catch (err) {
    return res.status(500).json({ error: "Lỗi server." });
  }
});
```

Middleware chung yêu cầu đăng nhập và trạng thái ACTIVE. Với thao tác sửa, route còn yêu cầu vai trò LEADER. Tuy nhiên, những kiểm tra đó không ràng buộc bản ghi được chọn với dòng họ của tài khoản. Nếu một hệ thống triển khai lưu nhiều dòng họ và có dữ liệu tương ứng, việc thay đổi định danh trong yêu cầu có nguy cơ cho phép truy cập chéo phạm vi. Cơ chế này phù hợp với nhóm CWE-639 [9].

Tác động tiềm tàng: lộ thông tin cá nhân hoặc sửa, xóa thành viên của dòng họ khác tùy quyền route. Mức ưu tiên khắc phục cao trong mô hình nhiều dòng họ, nhưng chưa có bằng chứng khai thác trên một phiên bản đã triển khai. Cần kiểm tra tích hợp bằng hai tài khoản thuộc hai dòng họ riêng biệt trước khi khẳng định phạm vi tác động thực tế.

4.7. Vấn đề WEB-02 trong khởi tạo cơ sở dữ liệu

Mức bằng chứng: đọc mã nguồn và kiểm thử đơn vị với cơ sở dữ liệu giả.

Trong backend/src/db/schema.js, hàm runSchema tách chuỗi SQL theo dấu chấm phẩy rồi loại mọi phần bắt đầu bằng --:

```
const statements = SCHEMA_SQL
  .split(";")
  .map(s => s.trim())
  .filter(s => s.length > 0 && !s.startsWith("--"));

for (const sql of statements) {
  await conn.query(sql);
}
console.log("Schema created/verified");
```

Các lệnh tạo bảng chính trong chuỗi đều được đặt sau chú thích mở đầu --. Vì chú thích và câu SQL cùng nằm trong một phần sau khi tách, bộ lọc loại cả phần đó. Kiểm thử thực thi hàm gốc với kết nối giả ghi nhận:

```
{
  "test": "runSchema with mock DB",
  "base_statements_sent": 0,
  "queries": []
}
```

Dù không gửi câu lệnh tạo bảng chính nào, hàm vẫn in thông báo đã tạo hoặc kiểm tra schema. Kết quả không có nghĩa toàn bộ tệp schema.js không gửi SQL: một số bảng bổ sung và migration được gọi riêng ngoài runSchema. Vấn đề được xác nhận nằm ở danh sách câu lệnh chính của hàm này.

Ngoài ra, phần định nghĩa bảng tài khoản và thành viên trong tệp schema chưa có cột clanId, trong khi các route có sử dụng cột này. Đây là điểm cần đối chiếu với migration hoặc schema thực tế khi dựng lại ứng dụng. Báo cáo chưa chạy MySQL để kiểm tra trạng thái một cơ sở dữ liệu có sẵn.

Tác động: trên cơ sở dữ liệu mới, các bảng chính có nguy cơ không được tạo như dự kiến, ảnh hưởng trực tiếp đến khả năng cài đặt và chạy lại. Đây là lỗi triển khai và tính tái lập đã có bằng chứng ở cấp hàm; không gán một mức CVSS hay mã CWE không cần thiết.

5. Thảo luận và hạn chế

5.3. Những giới hạn của thực nghiệm

Chỉ chạy bộ ca cố định và một số ca biên bổ sung, chưa có chiến dịch fuzzing dài với corpus biến đổi tự động.

Chưa thực hiện kiểm thử tích hợp frontend, API và MySQL; không có số liệu tải, thông lượng hay mức sẵn sàng của ứng dụng.

Sanitizer chỉ quan sát các đường đi đã thực thi. Không có cảnh báo trong một lần chạy không chứng minh chương trình an toàn với mọi đầu vào.

Việc tắt phát hiện rò rỉ làm cho thực nghiệm không đánh giá memory leak.

Kích thước cấu trúc và mã kết thúc tiến trình phụ thuộc môi trường; cần ghi lại cấu hình khi tái lập trên hệ khác.

Trạng thái registry của bộ ca gốc được giữ giữa các ca, do đó kết quả phụ thuộc vào thứ tự và không tương đương 11 ca hoàn toàn cô lập.

Chưa kiểm tra tình huống malloc thất bại. Cả cấp phát ban đầu lẫn cấp phát lại đều gọi memset ngay sau malloc mà không kiểm tra kết quả, nên đây là hướng cần khảo sát tiếp.

Không áp dụng softsec-toolkit, SMT/BMC hoặc symbolic execution. Không đưa tên các kỹ thuật này vào phần thành tích như thể đã sử dụng.

Các đoạn mã khắc phục ở mục tiếp theo là đề xuất, chưa được áp dụng vào mã nguồn gốc hoặc kiểm thử hồi quy trong thực nghiệm này.

5.5. Hướng phát triển

Có thể phát triển harness libFuzzer nhận Data và Size, giải mã thành cấu trúc đầu vào có giới hạn, đặt lại trạng thái toàn cục giữa các lần chạy và lưu đầu vào gây lỗi. Corpus nên bao gồm độ dài âm, ranh giới bộ đệm, chuỗi thiếu ký tự kết thúc khi phù hợp với giao diện nhận dữ liệu, chỉ số ngoài miền và đồ thị có chu trình. Các đầu vào này phải được đưa vào qua bộ giải mã an toàn để phân biệt lỗi harness với lỗi mục tiêu.

Ở ứng dụng web, cần bổ sung bộ kiểm thử tích hợp dùng dữ liệu giả, kiểm tra phạm vi dòng họ, cập nhật quan hệ gia đình và quy trình khởi tạo. Mọi phép so sánh giữa kỹ thuật kiểm thử trong

tương lai nên dùng cùng mục tiêu, ngân sách và tập lỗi tham chiếu. Báo cáo hiện tại chưa có cơ sở để so sánh định lượng black-box và white-box fuzzing.

6. Kết luận

Báo cáo đã phân tích phiên bản cập nhật của hệ thống quản lý dòng họ và mô hình C gồm một module xử lý năm bước. Chiến dịch gồm 11 ca, đạt 16/16 nhãn trạng thái, 16/16 nhãn mệnh đề và 10/10 path khai báo. Đối chiếu độc lập xác nhận 11 trace baseline khớp chuỗi đặc tả tương ứng. Cấu trúc kết thúc sớm ở nhánh bất thường giúp thể hiện rõ điểm dừng của từng ca.

Tám nhãn "CRASH" và một nhãn "HANG" vẫn là các sự kiện do chương trình tự phân loại; baseline chạy cùng sanitizer kết thúc với mã 0. Kiểm thử bổ sung xác nhận lỗi độ dài audit âm còn tồn tại, điều kiện phát hiện thu hẹp số vẫn bỏ sót count 16384 và một số mệnh đề không khớp điều kiện thực thi. Phép đo mới ghi nhận sizeof(ClanMember) = 72 byte trên ARM64.

Bộ đo path chỉ so khớp trạng thái cuối, vì vậy cần bổ sung so sánh đầy đủ trace để bảo đảm tính đúng khi mô hình thay đổi. Mã web không thay đổi; nguy cơ thiếu ràng buộc dòng họ và lỗi lọc SQL trong runSchema vẫn cần xử lý theo mức bằng chứng đã nêu. Hướng hoàn thiện ưu tiên kiểm tra cận đầu vào, sửa oracle và đặc tả, xác thực path, bổ sung kiểm thử tích hợp và bảo đảm quy trình dựng cơ sở dữ liệu.

Phụ lục

Phụ lục B. Chạy lại thực nghiệm

Điều kiện: Python 3, Node.js và Clang có ASan/UBSan. Các kiểm tra đã thực hiện không cần cơ sở dữ liệu MySQL. Từ thư mục gốc của gói:

```
python3 tools/reproduce.py
python3 tools/validate_traces.py
node tools/check_schema.cjs > results/schema_unit.txt
```

Script đầu biên dịch, chạy baseline và các probe, lưu stdout/stderr cùng mã kết thúc. Script thứ hai so sánh toàn bộ 11 trace baseline với 10 chuỗi path trong source. Kiểm thử schema dùng kết nối giả và ghi số câu lệnh chính được gửi. Chạy riêng hai tình huống quan trọng sau khi biên dịch:

```
ASAN_OPTIONS=detect_leaks=0 results/probe audit -1
ASAN_OPTIONS=detect_leaks=0 results/probe path_mismatch
```

Lệnh audit dự kiến bị ASan chấm dứt. Lệnh path_mismatch là kiểm thử đơn vị bộ đo, chủ động tạo trace không đầy đủ nhưng cùng trạng thái cuối PATH_01. Baseline có thể được lưu riêng bằng chuyển hướng stdout; chương trình C không tự tạo tệp log:

```
ASAN_OPTIONS=detect_leaks=0 results/baseline > results/baseline.stdout.txt
```

Ứng dụng web vẫn chưa được chạy tích hợp. Backend đọc các biến DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET; frontend dùng VITE_API_URL, mặc định http://localhost:3001/api. Trước khi dựng trên cơ sở dữ liệu mới cần xử lý lỗi schema ở mục 4.7. Hướng dẫn npm trong README không được dùng làm bằng chứng ứng dụng đã chạy thành công.

Phụ lục C. Đối chiếu kết luận với bằng chứng

Nội dung kiểm tra	Tệp bằng chứng
11 ca, ba tỷ lệ độ phủ và các nhãn sự kiện	results/baseline.stdout.txt
Baseline kết thúc 0, stderr không có chẩn đoán	results/manifest.json, results/baseline.stderr.txt
Lỗi audit âm	results/audit_-1.stderr.txt

Các biên audit và fund	Các tệp results/audit_*.stdout.txt, results/fund_*.stdout.txt
Sai lệch mệnh đề	results/bio.stdout.txt, results/spouse_ 14.stdout.txt, results/lifecycle_3.stdout.txt
Kích thước cấu trúc 72 byte	results/size.stdout.txt
Bộ nhận dạng chỉ xét trạng thái cuối	results/path_mismatch.stdout.txt
11 trace thực tế khớp toàn bộ chuỗi khai báo	results/trace_validation.json
Lỗi khởi tạo bảng chính	results/schema_unit.txt
Quy mô và dấu kiểm tra từng tệp nguồn	results/source_inventory.json
Nội dung bốn sheet tài liệu tham khảo	results/workbook_reference.json